Indian Beans Payments API — Korean Integration Guide

Accept KakaoPay payments on your Korean site · API host: `https://indianbeans.com` · Rev. 2026-10-11

Two integration options

Pick whichever matches your business flow. The API is the same for both; one request field (`qr_mode`) switches the UX.

Option	Use when	Customer experience
B · QR	Walk-in / POS, retail counter, invoice sent to a client.	Merchant shows one QR. Customer scans with phone camera → KakaoPay app opens pre-filled → tap Pay. No email / no form.
A · iframe	Customer buys from your Korean website / web app.	KakaoPay flow runs inside an `<iframe>` on your page. Your parent window receives a `postMessage` on success.

Prerequisites (both options)

1. An Indian Beans account at `https://indianbeans.com/signup`.
2. An API key: **Portal** → **Settings** → **API Keys** → **Create new key**. Copy it immediately — the full value is shown once.
3. A webhook endpoint on your server (optional but recommended) that we POST to on payment success / failure.

Option B QR walk-in flow

You call the API once per customer and get back a QR code. Show the QR to the walk-in customer; they scan it with their phone camera, KakaoPay opens with the amount pre-filled, they tap Pay. The customer never fills out a form.

Step B1 · Create the payment (your server)

```
curl -X POST https://indianbeans.com/api/public/v1/payments \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "amount": 1000,
    "currency": "KRW",
    "payment_methods": ["kakao_pay"],
    "qr": true,
    "qr_mode": "auto_kakao",
    "customer": { "email": "pos@your-site.co.kr" },
    "description": "Order #YOUR_ORDER_ID",
    "webhook_url": "https://your-site.co.kr/webhooks/hub-paid",
    "metadata": { "order_id": "YOUR_ORDER_ID" }
  }'
```

> **Amounts in native units for KRW / JPY / VND.** Pass the real amount — `1000` means ₩1,000, `5000` means ₩5,000. No multiplication by 100. (For USD / EUR / CAD, pass cents as usual: `500` = $5.00.)

Response (201 Created):

```
{
  "id":           "cmg...",
  "status":       "created",
  "amount":       1000,
  "currency":     "krw",
```

```
  "qr_mode":      "auto_kakao",
  "checkout_url": "https://indianbeans.com/l/AbCdEfGh?auto=kakao",
  "qr_data_url":  "...",
  "link_slug":    "AbCdEfGh",
  "created_at":   "2026-10-11T12:34:56Z"
}
```

Step B2 · Display the QR to the customer

The `qr_data_url` is a ready-to-render PNG. Drop it into your POS screen, invoice email, or printed receipt:

```
<img src="QR_DATA_URL" alt="Scan to pay with KakaoPay" width="300" height="300">
```

What happens when the customer scans:

1. Customer opens their phone camera and points at the QR.
2. Phone detects the link and offers to open it — customer taps.
3. Phone browser shows a brief *"Opening KakaoPay…"* splash (~1 second).
4. KakaoPay app opens with the amount, merchant and reference pre-filled.
5. Customer taps **Pay** inside KakaoPay. Receipt is shown in the KakaoPay app.

Step B3 · Receive the webhook on your server (recommended)

On payment success/failure we POST your `webhook_url` with a JSON body. Verify, mark the order paid, respond `200 OK`.

```
POST https://your-site.co.kr/webhooks/hub-paid
Content-Type: application/json

{
  "event":    "payment.succeeded",
  "payment": { "id": "cmg...", "amount": 1000, "currency": "krw",
               "status": "paid", "method": "kakao_pay" },
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}
```

> **Why email isn't required:** walk-in customers don't supply one. KakaoPay sends its own receipt in-app, so we only need an email for the server-side invoice record. The one in `customer.email` above should be a merchant-side address (e.g. POS, accounting) — never a fabricated customer email.

B2B Tax Exemption NEW · 2026-10-11

By default KakaoPay payments are taxed at the standard Korean rate. For business buyers who provide a valid **사업자등록번호 (Business Registration Number)**, you can zero-rate the invoice by including the business details in the QR URL. Paddle validates the number and applies the exemption automatically.

How it works

The response's `checkout_url` is a QR URL ending with `?auto=kakao`. For a business buyer, append two extra query parameters:

- `bizName` — URL-encoded business name
- `taxId` — Korean 사업자등록번호 (with or without dashes)

Example — individual (standard, taxed):

```
https://indianbeans.com/l/AbCdEfGh?auto=kakao
```

Example — business (tax-exempt):

```
https://indianbeans.com/l/AbCdEfGh?auto=kakao
  &bizName=Customer%20Company%20Ltd
  &taxId=123-45-67890
```

Two ways to add it

1. Your server appends it before generating the QR (recommended):

```
// Mint the payment link
const res = await fetch('https://indianbeans.com/api/public/v1/payments', { ... });
const { checkout_url } = await res.json();

// For a business buyer, append their details to the URL
const businessUrl = `${checkout_url}` +
  `&bizName=${encodeURIComponent(business.name)}` +
  `&taxId=${encodeURIComponent(business.taxId)}`;

// Generate the QR from businessUrl instead of checkout_url
const qrPng = await generateQR(businessUrl);
```

2. Hand-coded QR for a single long-standing business customer: generate the URL once with the business params baked in, print the QR, display it at their office. Every scan from that QR is tax-exempt. For your other walk-in customers, keep using the standard QR without the extra params.

> **Important: the `qr_data_url` in the API response is NOT the right QR for B2B.** Our API generates it from the base `checkout_url` only — it has no way to know what business params you're going to append. For a B2B QR you must:
>
> 1. Call the API as usual and keep the `checkout_url` value.
> 2. Append `&bizName=...&taxId=...` to that URL.
> 3. Encode the **final URL** as a QR yourself (any QR library, or an online generator such as `api.qrserver.com/v1/create-qr-code/`).
>
> The `qr_data_url` field is only useful for the standard consumer flow.

Important rules

> **Only use the business QR for actual business buyers.** Using tax-exempt URLs for consumer payments creates incorrect records in Korean tax law. One business → one dedicated QR. Walk-in consumers → the standard QR

without `bizName` / `taxId` .

> **Validation is silent.** Paddle validates the tax ID but does not error when it's invalid — it silently applies the standard tax instead. Always verify the first payment from any new business QR to confirm the invoice is zero-rated before committing to it in production.

Both parameters must be present together. Passing only `bizName` or only `taxId` is ignored — the flow falls back to the standard taxed payment.

Existing QRs

No change required. Any QR already issued without the business params continues to work exactly as before. The B2B exemption is purely additive — enable it only for the specific business customers who have provided their tax ID.

Option A Iframe embed flow

For when the customer is already on your Korean website buying something. The KakaoPay checkout renders inside an `<iframe>` you drop on your page; your parent window gets a `postMessage` on success.

Step A1 · Create the payment (your server)

```
curl -X POST https://indianbeans.com/api/public/v1/payments \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "amount": 1000,
    "currency": "KRW",
    "payment_methods": ["kakao_pay"],
    "customer": {
      "email": "buyer@your-site.co.kr",
      "name":  "홍길동"
    },
    "description": "Order #YOUR_ORDER_ID",
    "webhook_url": "https://your-site.co.kr/webhooks/hub-paid",
    "metadata": { "order_id": "YOUR_ORDER_ID" }
  }'
```

Note: no `qr_mode` field — that's what switches to Option B. For the iframe flow omit it.

Response includes an iframe-ready URL:

```
{
  "id":           "cmg...",
  "checkout_url": "https://indianbeans.com/l/AbCdEfGh",
  "embed_url":    "https://indianbeans.com/l/AbCdEfGh?embed=1",
  "link_slug":    "AbCdEfGh"
}
```

Step A2 · Embed on your customer-facing page

```
<iframe
  id="hub-payment"
  src="EMBED_URL"
  width="100%"
  height="700"
  frameborder="0"
  allow="payment"
  style="border:0; max-width:480px; display:block; margin:0 auto;"
></iframe>

<script>
  window.addEventListener('message', function(e) {
    if (e.origin !== 'https://indianbeans.com') return;   // always verify origin
    if (e.data && e.data.type === 'payment:succeeded') {
      document.getElementById('hub-payment').style.display = 'none';
      document.getElementById('thank-you').style.display = 'block';
      fetch('/mark-paid?invoice=' + encodeURIComponent(e.data.invoiceId));
    }
  });
</script>
```

Step A3 · Receive the webhook

Same shape as Option B. See **Step B3**.

Why both postMessage + webhook? The `postMessage` is instant UI feedback for the customer in the iframe. The webhook is the authoritative server-to-server confirmation — always mark orders paid from the webhook, never only from the browser message.

Reference

Request fields — POST /api/public/v1/payments

Field	Type	Notes
`amount`	integer	**Zero-decimal currencies (KRW, JPY, VND):** native units — `1000` means ₩1,000. **Sub-unit currencies (USD, CAD, EUR, INR):** smallest unit — `500` means $5.00. **Required.**
`currency`	string	ISO 4217 code. KRW, USD, CAD, EUR, etc.
`payment_methods`	string[]	`["kakao_pay"]`. (Also supports `["paypal"]`, `["alipay"]`.)
`qr`	boolean	`true` → `qr_data_url` included in response.
`qr_mode`	string	`"auto_kakao"` → Option B (QR walk-in). Omit for Option A (iframe).
`customer.email`	string	Required. For Option B this is a merchant-side address — not shown to the customer.
`customer.name`	string	Optional. Supports Korean characters.
`description`	string	≤ 500 chars. Shown on the Paddle hosted checkout page.
`webhook_url`	string	HTTPS. Where we POST on payment events.
`metadata`	object	Any JSON. Echoed in the webhook payload.
`max_uses` NEW	integer	How many times this payment link can be used (same amount). Default **100**. Pass `1` for strict single-use (one payment per API call). Pass `0` or `null` for unlimited. Clamped to 1–10000 otherwise.

Reusability. One API call returns one link that can be paid up to `max_uses` times at the same amount. For walk-in POS where each sale has a different amount, keep the one-call-per-sale pattern (default 100 is still safe — unused slots simply expire). For a dedicated business QR where the customer pays the same amount repeatedly, 100 lets you reuse the same QR across all their payments.

QR URL query parameters (append to `checkout_url`)

Parameter	When used	Notes
`auto=kakao`	Option B (QR walk-in)	Set automatically by our API when `qr_mode: "auto_kakao"`.
`embed=1`	Option A (iframe)	Hides page chrome, enables `postMessage`.
`bizName` NEW	B2B tax exemption	URL-encoded business name. Must be present together with `taxId`.
`taxId` NEW	B2B tax exemption	Korean 사업자등록번호 (BRN). Dashes optional. Paddle validates and zero-rates when recognised.

Response fields

Field	Appears for	Use for
`id`	Both	Internal reference (an invoice or link id).
`checkout_url`	Both	Full-page checkout. Share via email, SMS, or print. For Option B this includes `?auto=kakao`.
`embed_url`	Option A	iframe `src`. Chrome stripped, `postMessage` on paid.
`qr_data_url`	Both (when `qr: true`)	Inline PNG QR of `checkout_url`. Scan with phone camera.
`qr_mode`	Option B only	Echoes `"auto_kakao"` so you can confirm the mode.
`link_slug`	Both	The short slug portion of `checkout_url`.

postMessage event (Option A only — iframe → parent)

```
{
  "type":          "payment:succeeded",
  "invoiceId":     "cmg...",
  "invoiceNumber": "INV-2026-0042",
  "amountCents":   1000,
  "currency":      "KRW"
}
```

Webhook body (both options)

```
POST https://your-site.co.kr/webhooks/hub-paid
Content-Type: application/json

{
  "event":    "payment.succeeded" | "payment.failed",
  "payment": {
    "id":       "cmg...",
    "amount":   1000,
    "currency": "krw",
    "status":   "paid" | "failed",
    "method":   "kakao_pay"
  },
  "customer": { "email": "...", "name": "..." },
  "metadata": { "order_id": "YOUR_ORDER_ID" }
}
```

Testing checklist

Option B — QR flow

- POST the request. Response **201** with `qr_data_url` and `qr_mode: "auto_kakao"`.
- Open `checkout_url` in a desktop browser — *"Opening KakaoPay…"* splash renders, Paddle checkout follows.
- Save `qr_data_url` as a PNG (decode the base64). Display it on screen or print it.
- Scan with a Korean phone camera that has the KakaoPay app installed. KakaoPay should open with the payment pre-filled.
- Complete a test payment (KRW minimum ≈ ₩1,000). Confirm your webhook endpoint received the `payment.succeeded` POST.

Option A — iframe flow

- POST the request. Response **201** with `embed_url`.
- Open `embed_url` directly in a browser — bare-chrome checkout loads.
- Drop the iframe snippet onto a test page on your site. Attach the `message` listener.
- Complete a test payment with a real Korean phone + KakaoPay. Confirm your parent-window listener fires and your webhook receives the event.

B2B tax exemption NEW

- Generate a QR URL with `?auto=kakao&bizName=TestBiz&taxId=123-45-67890`.
- Scan and complete a small test payment with a real Korean 사업자등록번호 you control.
- Verify the invoice line in the KakaoPay app receipt shows the amount **without** VAT added. If VAT still appears, the tax ID wasn't validated — double-check the number and try again.

Minimum amounts & limits

- **KRW minimum per transaction:** ~₩1,000. Smaller amounts are rejected by Paddle with a "transaction balance less than minimum" error.
- Amounts are integers. For zero-decimal currencies (KRW, JPY, VND) pass native units — ₩1,000 is `1000`. For sub-unit currencies (USD, EUR, CAD, INR) pass the smallest unit — $5.00 is `500`.
- One payment request returns one link that defaults to **100 payments max** at the same amount. Override with the `max_uses` field (`1` for strict single-use, `0` for unlimited).

> **Security:** never ship your API key in frontend code. Always call `/api/public/v1/payments` from your backend. The returned `checkout_url` / `embed_url` / `qr_data_url` are safe to send to the browser — each is a short-lived single-use session.

Support

Technical questions: `hello@indianbeans.com` · Response within one business day. Include your API key prefix (first 10 chars) and the `id` from the response.